

Eolas User Verification Test Plan

Document status: Draft 1.0

Date: 1 September 2026

Purpose: Thorough black-box user testing by an independent verification team.

1. Purpose

This document defines the user-level verification approach for Eolas. It is intended for a verification team that tests the product as an end user rather than by inspecting implementation details.

The objectives are to establish that:

- core user journeys work from start to finish;
- data entered by users is retained accurately and remains understandable;
- navigation and terminology are clear without developer guidance;
- invalid, incomplete and unexpected input is handled safely;
- documents and reports are accurate and useful;
- privacy-sensitive information is not exposed unnecessarily;
- the application behaves predictably after restart, update and failure;
- accessibility and usability are adequate for normal use;
- previously working behaviour has not regressed.

This is a repeatable release-verification document. Tests that are not implemented in a particular build should be recorded as **Not Available** rather than omitted.

2. Verification principles

1. Test from the user's point of view.
2. Use a clean test environment wherever practical.
3. Do not rely on developer knowledge to complete a workflow.
4. Record actual results, not only Pass/Fail.
5. Capture evidence for every failure and for key release-critical passes.
6. Repeat high-risk tests after fixes.
7. Preserve the test data set used for a release so results can be reproduced.
8. Where the expected behaviour is unclear, raise a specification/query defect rather than inventing an expected result.

3. Test roles

Role	Responsibility
Verification Lead	Owns the test cycle, assigns tests, reviews evidence and recommends release acceptance.
Tester	Executes test cases exactly as a user would, records results and raises defects.
Product/Requirements Reviewer	Resolves ambiguities in intended behaviour.
Developer	Investigates defects and provides fixed builds; should not self-verify release-critical fixes where independent verification is possible.

Release Owner

Makes the final release decision using the verification report.

4. Test status values

Use only the following statuses:

- **PASS** – observed behaviour matches the expected result.
- **FAIL** – observed behaviour differs from the expected result.
- **BLOCKED** – the test cannot be completed because of another defect or environmental issue.
- **NOT AVAILABLE** – the feature is planned but not present in this build.
- **NOT APPLICABLE** – the test genuinely does not apply to this build/platform.
- **NOT RUN** – the test has not yet been executed.

A test should not be marked PASS if a workaround was required unless the workaround is part of the documented user experience.

5. Entry criteria

Before a formal verification cycle begins:

- the build/version under test is uniquely identifiable;
- release notes or a change summary are available;
- installation/start instructions are available;
- known limitations are documented;
- a clean test data set is available;
- the verification team has access to supported platforms;
- release-critical automated tests have completed successfully, where applicable;
- no known defect prevents basic application startup.

6. Exit criteria

A release candidate is ready for verification sign-off when:

- all release-critical tests have been run;
- there are no unresolved Critical defects;
- there are no unresolved High defects unless explicitly accepted by the release owner;
- all failed regression tests have been investigated;
- data integrity tests pass;
- install/start/restart tests pass;
- report/export tests relevant to the release pass;
- the verification lead has produced a test summary.

7. Standard test environment record

Complete this once for each test environment.

Field	Value
Eolas version/build	
Git commit/tag if supplied	
Date tested	
Tester	

Operating system/version
Desktop environment
Screen resolution/scaling
Installation type Fresh / Upgrade
Test data set/version
Locale
Other relevant software
Notes

8. Standard evidence requirements

For failures record:

- test case ID;
- exact application build;
- test data used;
- numbered reproduction steps;
- expected result;
- actual result;
- screenshot or screen recording where useful;
- generated/exported file where relevant;
- logs if the application exposes them;
- whether the issue is repeatable;
- severity recommendation.

Do not include genuine family, banking or other private information in defect evidence. Use synthetic test data.

9. Core synthetic test data

Create a reusable fictional data set large enough to expose relationship and navigation errors.

Suggested data:

- **Clann:** Test Family
- **Primary household:** Oak House
- **Secondary household:** River Cottage
- **People:** Alex Test, Morgan Test, Jamie Test, Rowan Test
- One person living in the primary household
- One person living in the secondary household
- One family member living elsewhere
- One non-family household member if supported
- At least two bank accounts with different institutions
- Several standing orders and direct debits
- At least one uploaded statement/document
- At least one missing/incomplete record
- At least one archived/closed item if supported
- Dates spanning past, present and future

All names, account identifiers and documents must be fictitious.

10. Test execution record

For each test record:

Field	Entry
Test ID	
Build	
Tester	
Date	
Status	PASS / FAIL / BLOCKED / NOT AVAILABLE / NOT APPLICABLE / NOT RUN
Actual result	
Evidence reference	
Defect reference	
Notes	

11. Test cases

A. Installation, launch and application lifecycle

UV-001 – Fresh installation / first launch

Priority: Critical

Installation Instructions

1. Open: <https://github.com/Glawster/clannEolas/releases/tag/v0.1.0-macos-test1>
2. Under Assets, download: ClannEolas-0.1.0-macos-arm64.pkg
3. Save it to Downloads.
4. Optional but recommended: verify the checksum in Terminal:

```
shasum -a 256 ~/Downloads/ClannEolas-0.1.0-macos-arm64.pkg
```
5. Confirm the result is:

```
6e82503a76a0ef4006e1f9c1b674b4fd521a9f2d2b35a2b43173d6003792ccb0
```

Preconditions: No existing Eolas user data or configuration on the test machine.

Steps: 1. Install or start Eolas using the documented user procedure. 2. Launch the application. 3. Observe the splash/startup experience. 4. Continue to the first usable screen.

Expected result: - Installation/startup instructions are sufficient. - The application starts without an unhandled error. - Startup information is understandable. - The first usable screen is reached without developer intervention. - No misleading sample/private data is shown.

UV-002 – Normal close and restart

Priority: Critical

1. Enter or modify a small amount of test data.

2. Close Eolas using the normal application close action.
3. Relaunch Eolas.
4. Reopen the modified data.

Expected result: The application closes cleanly, restarts normally and retains committed data exactly once.

UV-003 – Repeated start/close cycle

Priority: High

Repeat launch and normal close five times.

Expected result: No progressive errors, duplicate records, prompts, corruption or unexpected state changes occur.

UV-004 – Unexpected termination recovery

Priority: Critical

1. Make a data change.
2. Simulate an unexpected application termination at a safe point.
3. Restart the application.
4. Inspect stored data and application state.

Expected result: Previously committed data remains valid. Partially completed work is either recovered safely or clearly not saved. The application does not become unusable.

UV-005 – Upgrade existing data

Priority: Critical

Where upgrade testing is supported: 1. Install a previous supported version. 2. Create representative data. 3. Upgrade to the candidate build. 4. Open and inspect every major data area.

Expected result: Existing data remains readable and relationships, documents and status information remain intact.

B. First-use experience and navigation

UV-010 – First-use comprehension

Priority: High

Give the application to a tester who has not used the build before. Without verbal coaching, ask them to identify: - what Eolas is for; - where to begin; - how to add the first family/clann/household information; - how to return to the main screen.

Expected result: The tester can identify the intended starting point and basic navigation without developer assistance.

UV-011 – Main navigation

Priority: Critical

Visit every top-level navigation destination and return to the start/dashboard after each.

Expected result: Every item opens the expected area; current location is clear; no dead-end screen exists.

UV-012 – Back/cancel behaviour

Priority: High

On create/edit screens, use Back, Cancel, close-window and equivalent actions.

Expected result: Behaviour is consistent. Unsaved changes are not silently discarded where loss would be surprising.

UV-013 – Keyboard navigation

Priority: High

Navigate primary workflows using keyboard controls where expected.

Expected result: Focus order is logical, controls can be reached, focus is visible and keyboard activation works.

UV-014 – Terminology consistency

Priority: Medium

Check labels for Clann, household, family member, person, account, document, review and other core concepts throughout the application.

Expected result: The same concept is named consistently and distinctions such as family versus household are understandable.

C. Clann, household and people management

UV-020 – Create a new Clann

Priority: Critical

Create the synthetic Test Family/Clann from an empty application.

Expected result: A single new Clann is created, shown in the expected location and remains after restart.

UV-021 – Create a household

Priority: Critical

Add Oak House with representative address/contact fields.

Expected result: The household is stored once, displayed accurately and linked to the intended Clann.

UV-022 – Add multiple households

Priority: High

Add River Cottage as a second household.

Expected result: Both households remain distinct and can be individually viewed and edited.

UV-023 – Add people

Priority: Critical

Add all synthetic people with different relationships and household membership.

Expected result: Each person is represented once and all entered fields are preserved accurately.

UV-024 – Person in a different household

Priority: High

Associate a family member with the secondary household.

Expected result: The person remains part of the correct Clann/family relationship while their household association is represented correctly.

UV-025 – Family member living elsewhere

Priority: High

Create a family member who is not part of either household.

Expected result: The application does not force an incorrect household membership.

UV-026 – Non-family household member

Priority: High / if supported

Create a person who belongs to a household but is not a family member.

Expected result: Household and family relationships remain separate and accurate.

UV-027 – Edit person details

Priority: Critical

Change several fields, save, navigate away and restart.

Expected result: Only intended fields change and the new values persist.

UV-028 – Duplicate person handling

Priority: High

Attempt to create the same person twice or a near-duplicate.

Expected result: The system either prevents/warns about likely duplication or handles it in a documented, understandable way.

UV-029 – Delete/archive a person

Priority: Critical if supported

Attempt to delete/archive a person with linked records.

Expected result: The application protects linked data, warns about consequences and never leaves

broken references.

D. Forms and data validation

UV-030 – Required fields

Priority: Critical

Attempt to save each principal record type with required values omitted.

Expected result: Save is prevented where necessary and the user is told exactly what needs attention.

UV-031 – Invalid formats

Priority: High

Try invalid dates, malformed email addresses, invalid numeric values and overly long input.

Expected result: Invalid values are rejected or clearly flagged without losing unrelated valid input.

UV-032 – Boundary values

Priority: High

Test empty values, minimum/maximum sensible dates, long names, punctuation, apostrophes, hyphens and multi-line notes.

Expected result: Valid real-world input is retained without corruption or unexpected truncation.

UV-033 – Special characters

Priority: High

Enter names and notes containing accented characters and common Unicode characters.

Expected result: Text displays, saves, reloads, searches and exports correctly.

UV-034 – Unsaved changes

Priority: Critical

Modify data, then navigate away or close the application without explicitly saving.

Expected result: The behaviour is deliberate and consistent; potentially destructive loss is clearly communicated.

UV-035 – Repeated save

Priority: High

Press Save repeatedly or trigger save using both keyboard and button where possible.

Expected result: The record is not duplicated and no error occurs.

E. Dashboard, readiness and status information

UV-040 – Dashboard accuracy

Priority: Critical

Create a known set of complete and incomplete data.

Expected result: Dashboard counts/statuses correspond to the underlying records.

UV-041 – Readiness score calculation

Priority: Critical if present

Create records that deliberately satisfy and fail known readiness criteria.

Expected result: Readiness results are reproducible and explainable from the data.

UV-042 – Readiness guidance

Priority: High

Select an incomplete or warning state.

Expected result: The application explains what is missing and provides a clear route to correct it.

UV-043 – Readiness refresh

Priority: Critical

Correct an incomplete item and return to the dashboard/readiness view.

Expected result: Status updates correctly without stale or contradictory values.

UV-044 – No-data dashboard

Priority: High

Open the dashboard with a new empty data set.

Expected result: Empty states are helpful and do not look like an error.

F. Banking and financial-information records

These tests concern documentation and preparedness records, not movement of real money. Use synthetic data only.

UV-050 – Add a bank account record

Priority: Critical

Create a fictional current account with institution, account purpose, contact/cancellation guidance and relevant notes.

Expected result: Information is saved accurately and displayed in a form useful to another family member.

UV-051 – Multiple bank accounts

Priority: High

Add accounts at different institutions with similar account types.

Expected result: Accounts are distinguishable and no data is accidentally shared between them.

UV-052 – Standing orders

Priority: Critical if supported

Add several standing orders including payee/purpose, amount/frequency and cancellation guidance.

Expected result: Each is linked to the correct account and information remains understandable.

UV-053 – Direct debits

Priority: Critical if supported

Add several direct debits.

Expected result: Entries are linked correctly and cancellation/contact information can be located by a user unfamiliar with the account.

UV-054 – Account access/contact guidance

Priority: High

Review an account as if the tester were a family member dealing with an emergency.

Expected result: The screen makes clear whom to contact, what records exist and what actions are expected without exposing unnecessary secrets.

UV-055 – Closed/archived account

Priority: High if supported

Close/archive a test account that has linked standing orders/direct debits/documents.

Expected result: Historical information remains coherent and linked records are not orphaned.

G. Document and statement handling

UV-060 – Upload supported document

Priority: Critical

Upload a representative fictional bank statement/document.

Expected result: The document is accepted, associated with the intended record and can be located again after restart.

UV-061 – Incorrect file type

Priority: High

Attempt to upload an unsupported or misleading file.

Expected result: The application rejects it safely with an understandable message.

UV-062 – Duplicate document

Priority: High

Upload the same document twice.

Expected result: Duplicate handling is predictable and does not create silent confusion.

UV-063 – Document identification/extraction

Priority: Critical if implemented

Use a synthetic statement containing known institution/account/reference and transaction information.

Expected result: Extracted information matches the visible document. The user can distinguish extracted/assumed information from authoritative user-entered information.

UV-064 – Ambiguous document extraction

Priority: Critical if implemented

Use a deliberately ambiguous document.

Expected result: The application does not confidently invent unsupported facts; uncertainty is exposed and requires user review where appropriate.

UV-065 – Incorrect extraction correction

Priority: Critical if implemented

Correct an extracted value.

Expected result: The correction persists and is not unexpectedly overwritten.

UV-066 – Remove document

Priority: High

Delete/unlink a document where supported.

Expected result: The user is warned appropriately and linked structured data is handled according to the documented rule.

UV-067 – Large/multi-page document

Priority: High

Upload a realistic multi-page statement near expected size limits.

Expected result: The application remains responsive enough for normal use and the complete document remains accessible.

H. Search and retrieval

UV-070 – Search by person

Priority: Critical

Search for an exact and partial person name.

Expected result: Relevant records are returned and can be opened directly.

UV-071 – Search by account/institution

Priority: High

Search using institution/account descriptive terms.

Expected result: Relevant account records are returned without revealing more sensitive data than necessary.

UV-072 – Search by notes/content

Priority: Medium if supported

Search for a distinctive phrase stored in notes or indexed documents.

Expected result: Results are accurate and their context is clear.

UV-073 – No-result search

Priority: High

Search for a value that does not exist.

Expected result: A clear no-results state appears with a simple way to revise/clear the search.

UV-074 – Search after edit/delete

Priority: High

Edit, archive or delete an indexed record and repeat the search.

Expected result: Search results reflect current application state.

I. Annual review and maintenance workflow

UV-080 – Start annual review

Priority: Critical if implemented

Begin an annual review for a populated Clann.

Expected result: The application explains the review scope and guides the user through items requiring confirmation.

UV-081 – Complete review without changes

Priority: High

Confirm all still-valid data.

Expected result: Review completion/date/status is recorded without altering unrelated data.

UV-082 – Update during review

Priority: Critical

Change outdated information while performing the review.

Expected result: Revised values are stored and review status reflects the completed review.

UV-083 – Incomplete review

Priority: High

Exit part way through.

Expected result: Progress is either saved safely or clearly discarded according to documented behaviour.

UV-084 – Overdue review

Priority: High

Use test dates that cause a review to become due/overdue.

Expected result: The application communicates the state clearly and does not use misleading urgency.

J. Reports, print and export

UV-090 – Generate core report

Priority: Critical if implemented

Generate a report/handbook from the standard test data.

Expected result: The report contains the correct Clann/household/person information and no records from another test data set.

UV-091 – PDF export

Priority: Critical if implemented

Export a representative report to PDF and inspect every page.

Expected result: - export completes successfully; - text is readable; - tables do not clip/overlap; - headings are correctly ordered; - page breaks are sensible; - no internal/debug information appears; - sensitive fields follow the intended inclusion rules.

UV-092 – Report with missing data

Priority: High

Generate a report where some fields are intentionally incomplete.

Expected result: Missing data is represented clearly and is not replaced by misleading guessed values.

UV-093 – Long-content report

Priority: High

Use long names, notes and enough records to span several pages.

Expected result: Layout remains readable and content is not lost.

UV-094 – Export filename/location

Priority: Medium

Perform multiple exports.

Expected result: The user can identify where the file was saved and accidental overwriting is handled predictably.

K. Data integrity and persistence

UV-100 – Cross-screen consistency

Priority: Critical

Edit a record and inspect every screen/report where the value is shown.

Expected result: The same current value appears everywhere.

UV-101 – Relationship integrity

Priority: Critical

Move/change a person's household or linked account relationships.

Expected result: All affected views update correctly and no stale/orphan links remain.

UV-102 – Restart integrity

Priority: Critical

After a substantial test session, close and restart the application.

Expected result: Counts, relationships, documents and values match the pre-restart state.

UV-103 – Reopen test data repeatedly

Priority: High

Open/save the same data set repeatedly across several sessions.

Expected result: No duplicate, gradual corruption or formatting drift occurs.

UV-104 – Simultaneous/competing edit behaviour

Priority: High if applicable

Attempt the supported equivalent of opening/editing the same data from two application instances or processes.

Expected result: The application prevents corruption and communicates any lock/conflict clearly.

L. Privacy and security from the user's perspective

UV-110 – Sensitive-field visibility

Priority: Critical

Review all screens containing banking/contact/private information.

Expected result: Only information needed for the intended preparedness purpose is shown. Passwords, PINs, full authentication secrets or equivalent data are not encouraged or exposed.

UV-111 – Screen/report privacy

Priority: Critical

Inspect dashboard, search results, recent-item lists and exported reports.

Expected result: Sensitive details are not unnecessarily exposed in summaries.

UV-112 – Test-data separation

Priority: Critical

Create two independent Clanns/data sets if supported.

Expected result: Records and documents from one never appear in the other unless explicitly linked.

UV-113 – Removed data

Priority: High

Delete/archive records through supported user actions and revisit searches/reports.

Expected result: User-visible behaviour matches the stated retention model; deleted data does not unexpectedly remain active.

UV-114 – Application/log error privacy

Priority: High

Trigger safe validation errors and inspect any user-visible diagnostics/log export.

Expected result: Messages are useful without exposing secrets or unrelated private content.

M. Error handling and recovery

UV-120 – Missing file/resource

Priority: High

Where safe, make an expected external document/resource unavailable and open its record.

Expected result: A clear error is shown and the rest of the application remains usable.

UV-121 – Read-only/unwritable location

Priority: High

Attempt an export or save to a location that cannot be written.

Expected result: The operation fails safely, gives a useful message and does not claim success.

UV-122 – Low-storage/export failure

Priority: High where practical

Simulate an export/storage failure in a controlled environment.

Expected result: Existing data remains intact and incomplete output is not presented as successful.

UV-123 – Invalid/corrupt imported document

Priority: Critical

Attempt to import a corrupt file with a supported extension.

Expected result: The application rejects it safely without crashing or damaging existing data.

UV-124 – Error recovery without restart

Priority: High

After a recoverable error, continue using other application areas.

Expected result: The application remains stable and does not require an unnecessary restart.

N. Accessibility and usability

UV-130 – Display scaling

Priority: High

Run at normal and increased desktop scaling/text size.

Expected result: Important controls and text remain visible, usable and not clipped.

UV-131 – Contrast/readability

Priority: High

Inspect text, disabled states, warnings, links and selected controls.

Expected result: Information remains legible and status is not communicated by colour alone.

UV-132 – Keyboard-only completion

Priority: High

Complete at least one full create/edit workflow without a pointing device.

Expected result: The workflow is practically usable and focus does not become trapped.

UV-133 – Labels and instructions

Priority: High

Ask a tester unfamiliar with the implementation to complete common tasks.

Expected result: Labels, prompts and validation messages use plain, actionable language.

UV-134 – Destructive-action clarity

Priority: Critical

Exercise delete/archive/remove actions using synthetic data.

Expected result: Consequences are clear before irreversible actions and confirmation wording identifies what will happen.

UV-135 – Empty states

Priority: Medium

View major screens with no records.

Expected result: Empty screens explain their purpose and offer an appropriate next action.

O. Performance and stability

User testing should measure perceived behaviour rather than micro-benchmarks.

UV-140 – Typical populated data set

Priority: High

Use a representative populated Clann and move repeatedly among dashboard, people, banking, search and reports.

Expected result: Interaction remains responsive enough that the tester does not believe the application has stopped responding.

UV-141 – Larger data set

Priority: High

Use a deliberately larger-than-normal synthetic data set.

Expected result: The application remains usable and does not produce timeouts, crashes or missing records.

UV-142 – Long session

Priority: High

Use the application continuously for an extended verification session involving create/edit/search/export operations.

Expected result: No progressive slowdown, memory-related symptoms or state corruption is observed.

UV-143 – Repeated report generation

Priority: Medium

Generate the same report several times after data changes.

Expected result: Each report reflects current data and no progressive failure occurs.

P. Regression and release-specific verification

UV-150 – Changed-feature verification

Priority: Critical

For every item in the release change summary: 1. identify the affected user journey; 2. run the new/changed behaviour; 3. run the nearest related existing behaviour.

Expected result: The change works and adjacent behaviour has not regressed.

UV-151 – Previous defect regression

Priority: Critical

Retest all defects marked fixed in the candidate build using the original reproduction steps.

Expected result: The original defect no longer reproduces and the fix has not introduced an obvious adjacent defect.

UV-152 – Critical-path smoke test

Priority: Critical

Minimum release smoke path: 1. launch; 2. open/create a Clann; 3. view/add a household; 4. view/add a person; 5. create/edit a key preparedness record; 6. search for it; 7. generate/export a report if supported; 8. close and restart; 9. verify the data remains correct.

Expected result: The complete path succeeds without a Critical or High defect.

12. Exploratory test charters

In addition to scripted cases, assign exploratory sessions.

Charter 1 – “A family member has to use Eolas unexpectedly”

Tester assumes they did not create the records themselves. Determine whether they can locate the key people, accounts, contacts and action guidance without help.

Charter 2 – “Everything is incomplete”

Populate partial data, missing dates, missing documents and uncertain information. Look for misleading confidence, broken screens and poor guidance.

Charter 3 – “Heavy editing”

Repeatedly create, edit, move, archive and reopen linked records. Look for stale values and relationship failures.

Charter 4 – “Document stress”

Use multi-page, duplicate, invalid, ambiguous and corrected synthetic documents. Focus on extraction confidence and traceability.

Charter 5 – “Unfamiliar user”

Give no product explanation beyond the application itself. Record terminology or workflows that require verbal coaching.

Charter 6 – “Recover from mistakes”

Deliberately press the wrong controls, enter invalid values, cancel actions and try to undo/recover. Look for silent data loss.

13. Defect severity

Severity	Definition
Critical	Data loss/corruption, serious privacy exposure, application unusable, or a release-critical journey cannot be completed with no reasonable workaround.
High	Major user function is wrong or unavailable; substantial risk of incorrect preparedness information; workaround is difficult or unsafe.
Medium	Function works imperfectly or causes meaningful confusion, but a reasonable workaround exists.
Low	Cosmetic, wording or minor usability issue with little operational impact.

Severity is about user impact, not how difficult the defect is to fix.

14. Defect report template

Title: [Area] concise observed problem

Build:

Test ID:

Severity:

Environment:

Test data:

Preconditions:

Steps to reproduce: 1. 2. 3.

Expected result:

Actual result:

Repeatability: Always / Intermittent / Once

Evidence: Screenshot, recording, exported file, log reference

Additional notes:

15. Verification cycle summary

At the end of the cycle record:

Measure	Result
Build tested	
Test dates	
Testers	
Total applicable tests	
PASS	
FAIL	
BLOCKED	
NOT AVAILABLE	
NOT RUN	
Critical defects open	
High defects open	
Medium defects open	
Low defects open	
Recommendation	Accept / Accept with known issues / Reject

Verification lead comments

Record: - principal risks; - areas not fully tested; - notable usability findings; - defects accepted for release; - follow-up testing required.

16. Release sign-off

Verification Lead: _____ **Date:** _____

Release Owner: _____ **Date:** _____

Decision: ACCEPT / ACCEPT WITH KNOWN ISSUES / REJECT

Conditions/notes:

Appendix A – Release-specific test additions

Each release should add tests for: - new requirements; - changed requirements; - fixed defects; - data migrations; - changed file formats; - changed reports; - platform-specific changes; - any area identified as high risk during development.

Permanent regression tests discovered during a release should be promoted into the main numbered test suite.

Appendix B – Verification-team working rules

- Never use live banking credentials or genuine authentication secrets.
- Prefer synthetic data throughout.
- Do not correct application data manually outside the user interface unless the test explicitly requires recovery testing.
- Do not mark a test PASS because the tester understands what the developer intended.
- Record confusing but technically functional behaviour as a usability defect when it creates a realistic risk of user error.
- Preserve failed exported files and screenshots with the defect record.
- Re-test Critical and High fixes independently where practical.
- Treat unexplained data changes as at least High severity until understood.